Лабораторная работа 5

Имитационное моделирование

Самарханова Полина Тимуровна

2026-04-17

Содержание I

1 Выполнение

Задания

- Создать рабочий каталог для всего курса.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
    - чистый код;

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Задания

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Цели и задачи

- Изучить моделирование задачи обедающих философов с использованием сетей Петри для анализа и предотвращения deadlock (взаимной блокировки). Реализовать две модели: классическую (с возможностью deadlock) и модифицированную (с арбитром), выполнить их стохастическое и детерминированное моделирование, визуализировать результаты и создать анимацию процесса.

Раздел 1

Выполнение

Код модели

Перехожу в каталог лабораторной работы 5 и создаю папку src и файл DiningPhilosophers.jl

```
$ cd ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/lab5

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ mkdir -p src

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ nano src/DiningPhilosophers.jl
```

Рисунок 1: создание файла

В открывшемся окне вставила код из методички(это только модуль с моделью, запуск не нужен)

```
        end
        t += dt
        if t <= tmax
            push!(times, t)
            push!(states, copy(u))
        end
    end

    df = DataFrame(time = times)
    for i = 1:net.n_places
        df[!, String(net.place_names[i])] = [s[i] for s in states]
    end
    return df
end

# Обнаружение deadlock

function detect_deadlock(df::DataFrame, net::PetriNet; tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])] for i = 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 && u_last[i] < -net.incidence[i, j] - tol
                can_fire = false
                break
            end
        end
        if can_fire
            return false
        end
    end
    return true
end

# Визуализация

function plot_marking_evolution(df::DataFrame, N::Int)
    plots = []
    for group in ["Think", "Hungry", "Eat", "Fork"]
        p = plot(xlabel = "Time", ylabel = group, title = "$group states")
        for i = 1:N
            col = "$group"_$i
            if col in names(df)
                plot!(df.time, df[!, col], label = "$group"_$i)
            end
        end
        push!(plots, p)
    end
    return plot(plots..., layout = (4, 1), size = (800, 1000))
```

Базовый эксперимент

Создаю папки plots data scripts и файл dining_philosophers.jl

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ mkdir -p plots data

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ mkdir -p scripts

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ nano scripts/dining_philosophers.jl
```

Рисунок 3: создание папок и файла

В открывшемся окне вставила код из методички

```
GNU nano 8.7                    scripts/dining_philosophers.jl
# scripts/dining_philosophers.jl

using DrWatson
using DataFrames, CSV, Plots

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(joinpath("data", "dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(joinpath("plots", "classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(joinpath("data", "dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(joinpath("plots", "arbiter_simulation.png"))
```

Установила необходимые пакеты

```
[131/d2d5] + OneTBB_jll v2022.0.0+1
      Info Packages marked with ^ and ⌅ have new versions available. Those with ^ may be upgradable,
ut those with ⌅ are restricted by compatibility constraints from upgrading. To see why use `status --o
tdated -m`
Precompiling packages ━━━
```

Запустила код и получила результаты.

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/stud
b5
$ julia scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Deadlock обнаружен: false

 Готово! Результаты сохранены в data/ и plots/
```

Рисунок 6: запуск

График классической симуляции

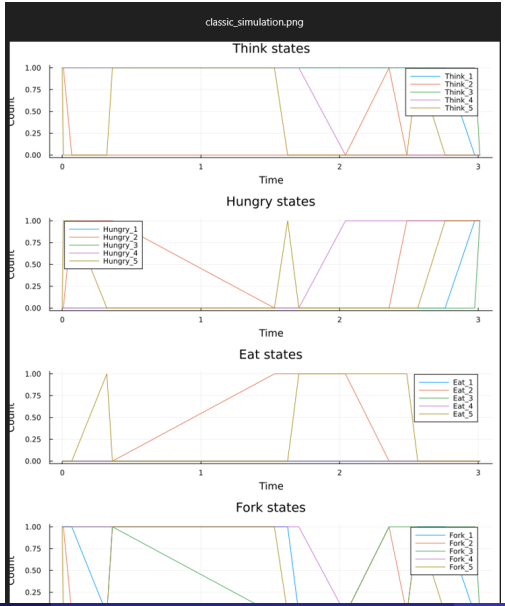
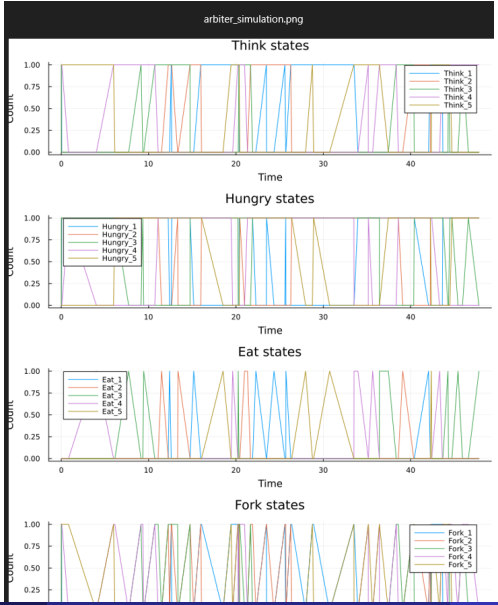


График арбитр симуляции



Анимация процесса

Создаю файл dining_philosophers_animation.jl

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ nano scripts/dining_philosophers_animation.jl
```

Рисунок 9: файл

В открывшемся окне вставила код из медодички

```
GNU nano 8.7                    scripts/dining_philosophers_animation.jl
# scripts/dining_philosophers_animation.jl

using DrWatson
using DataFrames, CSV, Plots, Random

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end
```

Установила необходимые пакеты

```
(@v1.12) pkg> add OrdinaryDiffEq Plots DataFrames Random LinearAlgebra CSV DrWatson
[ Info: Use `./Plots` to add or develop the local directory at `C:\Users\Polina Samarkhanova\work\study
\2026-1\2026-1==study--simulation-modeling\labs\lab5\Plots`.
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\Polina Samarkhanova\.julia\environments\v1.
12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\Polina Samarkhanova\.julia\environments\v1.
12\Manifest.toml`
```

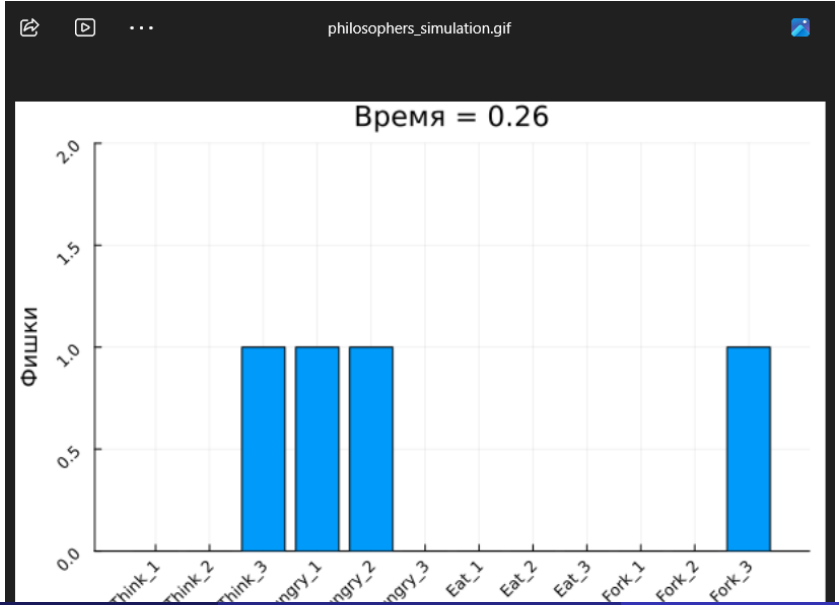
Рисунок 11: установка пакетов

Запустила код и получила результат - анимацию

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ julia scripts/dining_philosophers_animation.jl
[ Info: Saved animation to C:\Users\Polina Samarkhanova\work\study\2026-1\2026-1==study--simulation-mod
eling\labs\lab5\plots\philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 12: запуск

Просмотр результата(скрин анимации - видео на платформах)



Итоговый отчет

Создала файл dining_philosophers_report.jl

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/
b5
$ nano scripts/dining_philosophers_report.jl
```

Рисунок 14: файл

В открывшемся окне вставила код из медодички

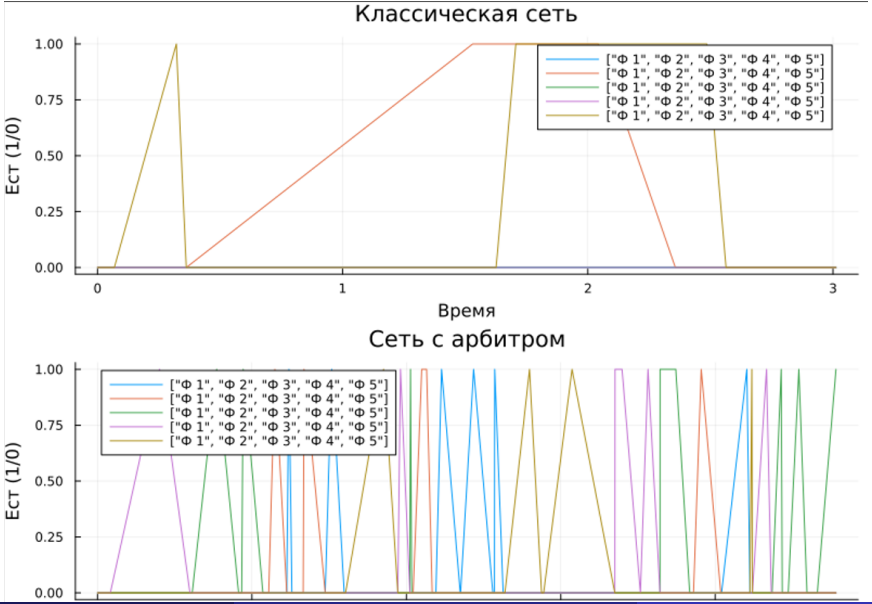
```
GNU nano 8.7                    scripts/dining_philosophers_report.jl
# scripts/dining_philosophers_report.jl

using DrWatson
using DataFrames, CSV, Plots

df_classic = CSV.read(joinpath("data", "dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(joinpath("data", "dining_arbiter.csv"), DataFrame)
N = 5

# Столбцы для состояния "Ест"
eat_cols = [Symbol("Eat_$i") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(joinpath("plots", "final_report.png"))
```

Запустила код и получила результат

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ julia scripts/dining_philosophers_report.jl
Отчёт сохранён в plots/final_report.png
```

Рисунок 16: запуск

Открыла графики с результатом



Спасибо за внимание !